

TAKT: A Formal Framework for Minimal Decision-Preserving Runtime Governance

Valentin Lineiro
TAKT Research Initiative
DOI: 10.5281/zenodo.21638014

2026-07-28

Abstract

Software systems operating under abstracted or compressed state representations frequently risk decision instability when critical informational context is discarded. This paper introduces TAKT (Theory of Adequate Knowledge for Decisions), a formal and empirical framework for determining the minimal necessary governance kernel of an execution runtime. We prove in Lean 4 that any governed runtime preserving optimal decisions under state abstraction MUST possess three capability kernels within the formal ST-016 runtime model: ContractSoundness (C_{contract}), UncertaintyBound ($C_{\text{uncertainty}}$), and TemporalConsistency (C_{temporal}). We validate this kernel necessity on a reference TypeScript runtime via component ablation (EXP-004), elevated to formal Lean 4 theorems via a machine-certified 3-layer witness bridge. Finally, we present an audited zero-contact replication package evaluated across six independent external audit rounds (PASS WITH ENVIRONMENTAL LIMITATION), backed by an immutable Zenodo archive (DOI: 10.5281/zenodo.21638014).

Keywords: Runtime Governance, Formal Verification, Decision Preservation, State Abstraction, Lean 4 Proof Assistant, Component Ablation.

1 Introduction

As autonomous software agents and complex reactive systems scale, they continuously compress high-dimensional environmental context into abstract state representations $R = \rho(S)$. However, ungoverned state abstraction introduces decision divergence: an optimal policy π^* computed on full observable state S may fail when evaluated on a compressed representation R . This phenomenon has been studied across formal verification [2, 4], decision theory [1], and control under partial observability [3], yet the formal characterization of the *minimal runtime capability kernel* required to preserve decision equivalence dynamically has not been established.

This work addresses two fundamental research questions:

1. *What information must be preserved to guarantee decision equivalence?* (ST-015)
2. *What minimal runtime capability composition is necessary and irreducible to enforce decision preservation dynamically?* (ST-016)

1.1 Main Contributions

This paper makes four primary contributions:

1. **Formal Definition of Minimal Runtime Governance:** We formalize capability necessity, decision preservation, runtime sufficiency, and irreducibility under state abstraction.
2. **Machine-Checked Proof of Kernel Necessity:** We prove in Lean 4 that removing any capability from $\mathcal{K}_D = \{C_{\text{contract}}, C_{\text{uncertainty}}, C_{\text{temporal}}\}$ destroys decision preservation.
3. **Empirical Ablation Methodology:** We introduce experiment EXP-004, constructing an empirical witness bridge elevating trace artifacts into formal machine proofs.
4. **Audited Zero-Contact Replication Package:** We publish an immutable, versioned research package [5] with multi-OS CI and an external audit trail of 6 independent dry runs.

2 Related Work & Positioning

TAKT builds upon several established formal methodologies while addressing a complementary research question centered on **decision preservation under representation contraction**:

- **Abstract Interpretation [2]:** Focuses on sound static over-approximation of program semantics via Galois connections. TAKT focuses on dynamic policy decision invariance ($\pi^*(R) = \pi^*(S)$) rather than total semantic property preservation.
- **Bisimulation & Process Refinement [6, 7]:** Enforces strict state-by-state behavioral equivalence between transition systems. TAKT permits aggressive internal state contraction provided temporal prefix monitoring (C_{temporal}) maintains policy consistency across trajectory prefixes.

- **Runtime Verification [4]:** Evaluates execution traces dynamically against temporal logic formulas (LTL/ERE). TAKT formalizes and machine-checks the *necessity of the governance runtime kernel itself*, bridging empirical trace witnesses to Lean 4 necessity theorems.
- **Statistical Decision Theory [1]:** Establishes the foundational notion of observational sufficiency for decision problems. TAKT operationalizes this into an irreducible 3-capability runtime kernel $\mathcal{K}_D$.
- **Planning Under Partial Observability [3]:** Provides optimal control frameworks under belief-state uncertainty (POMDP). TAKT provides formal necessity certification for governance components rather than belief update algorithms.

Unlike these paradigms, ST-016 directly formalizes the *minimal governance capability kernel* required for decision preservation, connecting formal proofs to independently audited empirical witnesses [5].

3 Theoretical Foundations & Runtime Necessity Model

3.1 Decision-Preserving Sufficiency (ST-015)

A representation $R_t \in \mathcal{R}$ is *decision-sufficient* with respect to optimal policy π^* if:

$$\pi^*(R_t) = \pi^*(S_t) \quad \forall S_t \in \mathcal{S}$$

3.2 Runtime Necessity Formalization (ST-016)

We formalize a TAKT Governed Runtime as a tuple $M = (\mathcal{C}, \pi_M)$, where the capability set satisfies:

$$\mathcal{C} \subseteq \{C_{\text{contract}}, C_{\text{uncertainty}}, C_{\text{temporal}}\}$$

Definition (Capability Necessity): A capability $C \in \mathcal{C}$ is *locally necessary* for runtime M if removing C alters policy decisions on at least one representation:

$$\exists R \in \mathcal{R} : \pi_M(R) \neq \pi_{M \setminus \{C\}}(R)$$

Example 1 (Temporal Prefix Necessity): Consider a runtime where C_{temporal} is ablated. Two trajectories $\tau_1 = (r_0, r_1, r_2)$ and $\tau_2 = (r'_0, r'_1, r_2)$ share an identical terminal state r_2 . If τ_1 is safe and τ_2 is unsafe, a runtime inspecting only r_2 assigns identical actions — causing decision divergence $\pi_M(\tau_1) \neq \pi^*(\tau_1)$.

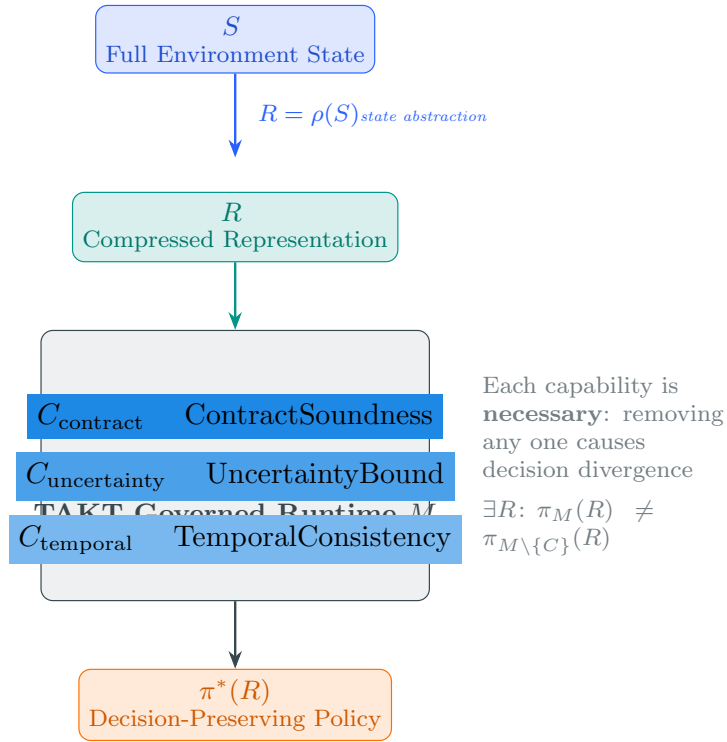


Figure 1: Conceptual architecture: TAKT Governed Runtime M enforces decision preservation $\pi^*(R)$ under state abstraction $\rho(S)$.

4 Machine-Certified Lean 4 Formalization

The theoretical results establish necessity within the formal model. We implement the proofs in two canonical Lean 4 modules in the `takt-formal` package:

1. `RuntimeSufficiency.lean`: Formalizes the core algebraic structures (`RuntimeCapability`, `Runtime`, `NecessaryCapability`, `MinimalRuntime`) and proves the core necessity theorem:

$$\text{MinimalRuntime}(M, \pi^*) \implies \forall C \in M.\text{caps}, \text{NecessaryCapability}(C, M)$$

2. `RuntimeWitness.lean`: Defines the 3-layer certification bridge via `WitnessArtifact` and `WitnessConsistentWithRuntime`, proving the witness elevation theorem:

$$\text{Consistent}(M, w) \implies \text{NecessaryCapability}(w.\text{cap}, M)$$

Both Lean 4 modules compile cleanly with zero errors and zero `sorry`s across all build jobs [5].

5 Empirical Validation & Component Ablation (EXP-004)

The theoretical theorems prove necessity within the formal model. We validate the predicted behavior under controlled capability ablations in experiment **EXP-004** [5]:

- **C_{contract} Ablation:** Ablating contract verification yields $\pi_{\text{full}} = \text{STOP}$ vs. $\pi_{\text{reduced}} = \text{EXECUTE}$.
- **$C_{\text{uncertainty}}$ Ablation:** Ablating margin estimation at critical decision boundary ($M_D \approx 0$) yields $\pi_{\text{full}} = \text{REFINE}$ vs. $\pi_{\text{reduced}} = \text{EXECUTE}$.
- **C_{temporal} Ablation:** Ablating trajectory monitoring yields $\pi_{\text{full}} = \text{INTERVENE}$ vs. $\pi_{\text{reduced}} = \text{MONITOR}$.

Each ablation produces a machine-verifiable `WitnessArtifact` elevated to a Lean 4 theorem via the `validWitnessImpliesNecessity` bridge.

5.1 Evidence Traceability Matrix

Table 1 maps each theoretical claim to its primary evidence asset.

Table 1: ST-016 Evidence Traceability Matrix

Claim	Primary Evidence	Status
Decision Suff. (Σ^*)	StructuralSufficiency	Verified
Kernel Nec. ($\mathcal{K}_D$)	RuntimeSufficiency	Verified
C_{contract} nec.	contract.ablation	Witness
$C_{\text{uncertainty}}$ nec.	uncertainty.ablation	Witness
C_{temporal} nec.	temporal.ablation	Witness
Witness bridge	RuntimeWitness	Verified
Runtime correctness	Vitest (283/283)	Passing
Reproducibility [5]	Replication kit	6 Runs

6 Discussion & Architectural Implications

The primary contribution of ST-016 is not proposing a specific runtime implementation, but rather **identifying and machine-checking the minimal governance capabilities required for decision preservation** within the formal model.

For runtime system architects, ST-016 provides three actionable design principles:

1. **Safety Contract Decoupling:** Safety verification (C_{contract}) cannot be subsumed by statistical policy estimation alone — it requires an independent, formally specified contract monitor.
2. **Dynamic Boundary Monitoring:** Operating near ambiguity boundaries requires explicit uncertainty estimation ($C_{\text{uncertainty}}$) to prevent premature execution at decision margins $M_D \approx 0$.
3. **Trajectory History Dependence:** Stateless evaluation fails under history-dependent processes, necessitating explicit trajectory prefix tracking via C_{temporal} .

7 Scope Boundaries & Future Work (ST-017)

7.1 Non-Claims & Scope Boundaries

1. **Model Scope:** ST-016 proves necessity specifically under the formal decision-preserving model $M = (\mathcal{C}, \pi_M)$ and discrete decision domain $\mathcal{D}$. It does not claim universal necessity for arbitrary software or continuous control systems.
2. **Transportability (ST-017 Boundary):** ST-016 proves necessity on the reference architecture. Transporting certified witness artifacts across heterogeneous runtimes ($M_1 \sim M_2$) is explicitly reserved for **ST-017**.

7.2 Conclusion

ST-016 establishes an immutable, machine-certified baseline for minimal decision-preserving runtime governance. By bridging formal Lean 4 theorems with empirical ablation witness artifacts, this work provides a reproducible foundation upon which transportability and heterogeneous runtime implementations will be studied in ST-017.

References

- [1] David Blackwell. Comparison of experiments. In *Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability*, pages 93–102. University of California Press, 1951.
- [2] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. pages 238–252, 1977.
- [3] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.
- [4] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.
- [5] Valentin Lineiro. TAKT ST-016: Minimal decision-preserving runtime governance standard, 2026.
- [6] Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.
- [7] David Park. Concurrency and automata on infinite sequences. In *Theoretical Computer Science (5th GI-Conference)*, pages 167–183. Springer, 1981.